scripts/10_build_ppt_assets.py

188 lines

```python
"""
10_build_ppt_assets.py
======================
Assembles everything needed to hand-build the submission deck into
`submission/presentation_assets/`.

Images are copied out of `figures/` with slide-mapped filenames
(`slide3_*.png`, `slide4_*.png`, ...) so it is obvious which asset belongs
where, and a couple of purpose-built composites are generated that do not
exist elsewhere:

* an architecture / data-flow diagram of the control loop
* a KPI strip for the results slide

Run after `run_all.py` (it needs the generated figures and result JSON).
"""

from __future__ import annotations

import json
import shutil
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "src"))

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch, FancyArrowPatch

from config import CONTROLLER, DATA_DIR, LIMITS

FIG = ROOT / "figures"
OUT = ROOT / "submission" / "presentation_assets"

INK = "#0f172a"
DIM = "#64748b"
BLUE = "#2563eb"
GREEN = "#16a34a"
AMBER = "#d97706"
RED = "#dc2626"
PURPLE = "#7c3aed"


# ---------------------------------------------------------------- diagram
def build_architecture_diagram(path: Path) -> None:
"""Control-loop architecture: what talks to what, once per hour."""
fig, ax = plt.subplots(figsize=(12, 5.7))
ax.set_xlim(0, 12); ax.set_ylim(0, 6); ax.axis("off")

def box(x, y, w, h, title, lines, fc, ec):
ax.add_patch(FancyBboxPatch((x, y), w, h, boxstyle="round,pad=0.06,rounding_size=0.12",
facecolor=fc, edgecolor=ec, linewidth=1.8))
ax.text(x + w / 2, y + h - 0.30, title, ha="center", va="top",
fontsize=11.5, fontweight="bold", color=INK)
for i, ln in enumerate(lines):
ax.text(x + w / 2, y + h - 0.62 - i * 0.29, ln, ha="center", va="top",
fontsize=8.8, color=DIM)

def arrow(x1, y1, x2, y2, label, color=BLUE, lx=None, ly=None):
ax.add_patch(FancyArrowPatch((x1, y1), (x2, y2), arrowstyle="-|>", mutation_scale=17,
linewidth=1.9, color=color))
ax.text(lx if lx is not None else (x1 + x2) / 2,
ly if ly is not None else (y1 + y2) / 2 + 0.16,
label, ha="center", va="bottom", fontsize=8.6,
```

```
color=color, fontweight="bold")

# --- top row: the forward path -------------------------------------
box(0.25, 2.70, 3.1, 1.95, "WELL / SIMULATOR",
["Reservoir → BHP → tubing", "→ WHP → choke → FLP", "→ manifold → separator",
"first-order lags + sensor noise"], "#eef2ff", BLUE)

box(4.45, 2.70, 3.1, 1.95, "IDENTIFIED MODEL",
["y(k+1) = y(k) + α(y_ss(u)+d−y)", "piecewise-linear y_ss(u)",
"τ per output, fitted from", "our own step tests"], "#f0fdf4", GREEN)

box(8.65, 2.70, 3.1, 1.95, "BRUTE-FORCE MPC",
["21 candidates (±5 %, 0.5 % grid)",
f"predict {CONTROLLER.horizon} h → filter → select",
"reject any predicted breach",
"deadband stops chatter"], "#faf5ff", PURPLE)

# --- constraint block feeds the MPC directly -----------------------
box(8.65, 0.85, 3.1, 1.30, "SAFE OPERATING ENVELOPE",
[f"WHP ≥ {LIMITS.whp_min:.0f} · FLP ≤ {LIMITS.flp_max:.0f} psi",
f"BHP ≥ {LIMITS.bhp_min:.0f} psi · |Δu| ≤ {CONTROLLER.max_move:.0f} %/step"],
"#fff7ed", AMBER)

arrow(3.40, 3.68, 4.40, 3.68, "Q, WHP, FLP, BHP", BLUE)
arrow(7.60, 3.68, 8.60, 3.68, "predicted\ntrajectories", GREEN, ly=3.76)
arrow(10.20, 2.20, 10.20, 2.65, "hard limits", AMBER, ly=2.24)

# --- feedback: routed ABOVE the row so it crosses nothing ----------
y_fb = 5.20
ax.plot([10.20, 10.20], [4.65, y_fb], color=PURPLE, lw=1.9)
ax.plot([10.20, 1.80], [y_fb, y_fb], color=PURPLE, lw=1.9)
ax.add_patch(FancyArrowPatch((1.80, y_fb), (1.80, 4.65), arrowstyle="-|>",
mutation_scale=17, linewidth=1.9, color=PURPLE))
ax.text(6.00, y_fb + 0.13, "next choke position u(k+1) — applied once per interval",
ha="center", va="bottom", fontsize=9, color=PURPLE, fontweight="bold")

ax.text(6.0, 5.78, "Control loop — executes once per hour (Ts = 1 h)",
ha="center", fontsize=13, fontweight="bold", color=INK)
ax.text(6.0, 0.18, "The controller only ever calls Q, WHP, FLP, BHP = simulator.step(u) "
"→ any simulator can be substituted with zero code changes",
ha="center", fontsize=9, color=DIM, style="italic")

fig.tight_layout()
fig.savefig(path, dpi=200, bbox_inches="tight", facecolor="white")
plt.close(fig)


# ------------------------------------------------------------------ KPI strip
def build_kpi_strip(path: Path, s: dict, r: dict) -> None:
meta = r["_meta"]
pct = 100 * r["C"]["final_Q_mean"] / meta["q_max_safe_ground_truth"]
kpis = [
(f"{meta['n_runs_per_scenario'] * 3}", "closed-loop runs", BLUE),
("0", "constraint violations", GREEN),
(f"{pct:.1f} %", "of max safe rate (C)", GREEN),
("19/19", "tests passing", BLUE),
(f"{meta['margin_cost_bblhr']:.1f}", "bbl/hr cost of safety", AMBER),
]
fig, ax = plt.subplots(figsize=(13, 1.65))
ax.set_xlim(0, len(kpis)); ax.set_ylim(0, 1); ax.axis("off")
for i, (big, small, col) in enumerate(kpis):
ax.add_patch(FancyBboxPatch((i + 0.06, 0.10), 0.88, 0.80,
boxstyle="round,pad=0.02,rounding_size=0.08",
facecolor=col, edgecolor="none"))
ax.text(i + 0.5, 0.60, big, ha="center", va="center",
fontsize=21, fontweight="bold", color="white")
ax.text(i + 0.5, 0.28, small, ha="center", va="center",
fontsize=9.5, color="white")
fig.tight_layout()
fig.savefig(path, dpi=200, bbox_inches="tight", facecolor="white")
```

```python
plt.close(fig)


# ---------------------------------------------------------------- main
def main():
OUT.mkdir(parents=True, exist_ok=True)
s = json.loads((DATA_DIR / "scenario_summary.json").read_text())
r = json.loads((DATA_DIR / "robustness_summary.json").read_text())

print("Generating purpose-built assets...")
build_architecture_diagram(OUT / "slide3_architecture_diagram.png")
print(" slide3_architecture_diagram.png")
build_kpi_strip(OUT / "slide4_kpi_strip.png", s, r)
print(" slide4_kpi_strip.png")

# slide-mapped copies of the pipeline figures
mapping = [
# numbering matches the FINAL deck, i.e. after the template's
# "IMPORTANT INSTRUCTIONS" slide has been deleted (6 slides total)
("step_gain_curve.png", "slide2_gain_curve.png"),
("step_identification.png", "slide3_step_tests.png"),
("model_validation.png", "slide3_model_validation.png"),
("dashboard/dash_reasoning.png", "slide3_mpc_reasoning.png"),
("robustness.png", "slide4_robustness.png"),
("scenarios_compact.png", "slide5_all_scenarios.png"),
("dashboard/dash_scenarioC_light.png", "slide5_dashboard_light.png"),
("dashboard/dash_scenarioC_dark.png", "slide5_dashboard_dark.png"),
("reference_vs_standin.png", "slide6_reference_vs_standin.png"),
("reference_dataset.png", "slide6_reference_dataset.png"),
("scenario_A.png", "appendix_scenario_A_full.png"),
("scenario_B.png", "appendix_scenario_B_full.png"),
("scenario_C.png", "appendix_scenario_C_full.png"),
]
print("\nCopying slide-mapped figures...")
missing = []
for src, dst in mapping:
p = FIG / src
if p.exists():
shutil.copy2(p, OUT / dst)
print(f" {dst}")
else:
missing.append(src)
if missing:
print(f"\n NOTE: not found (run run_all.py first): {missing}")

print(f"\nAll assets in {OUT.relative_to(ROOT)}/ ({len(list(OUT.glob('*.png')))} images)")


if __name__ == "__main__":
main()

```